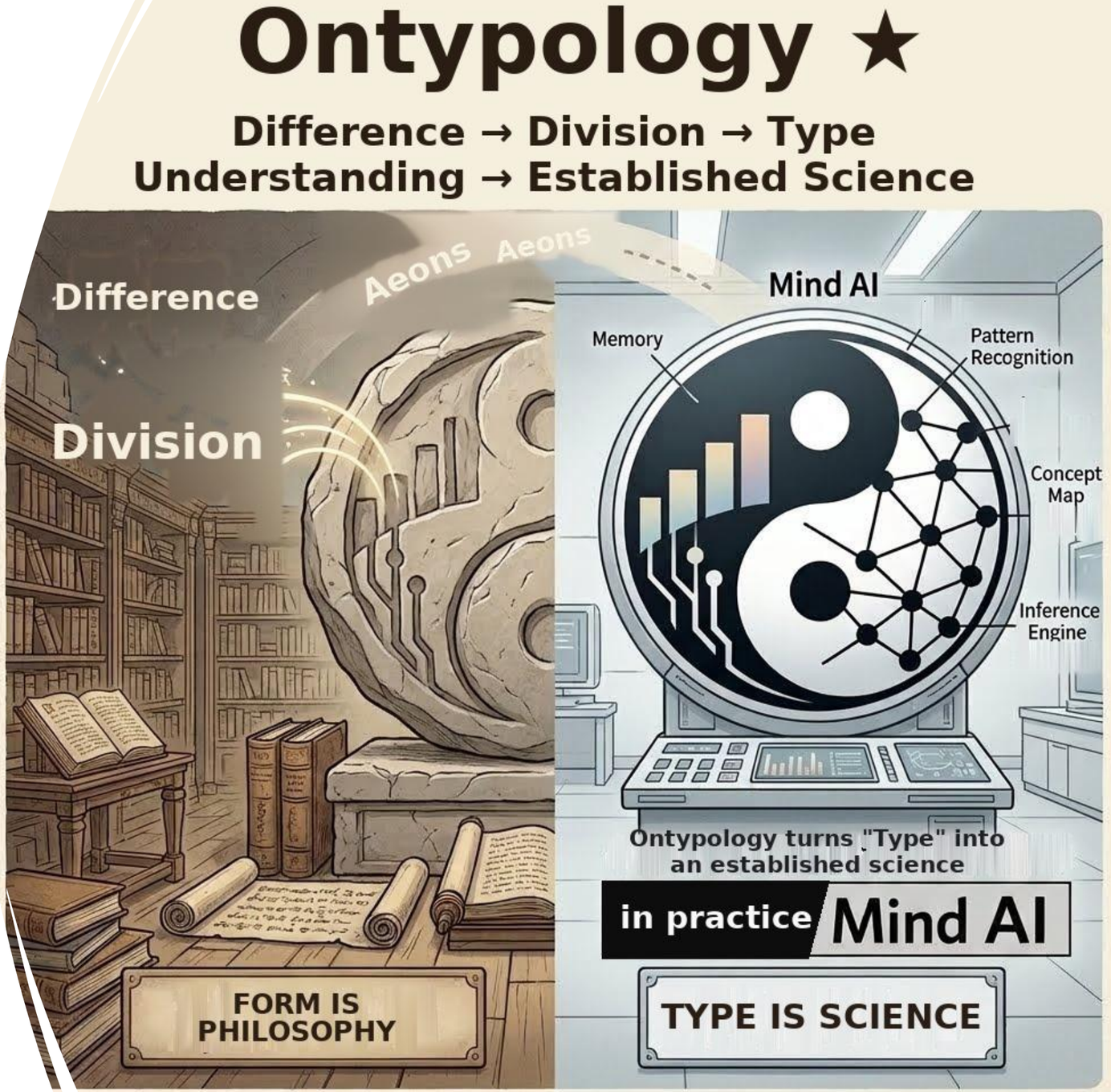
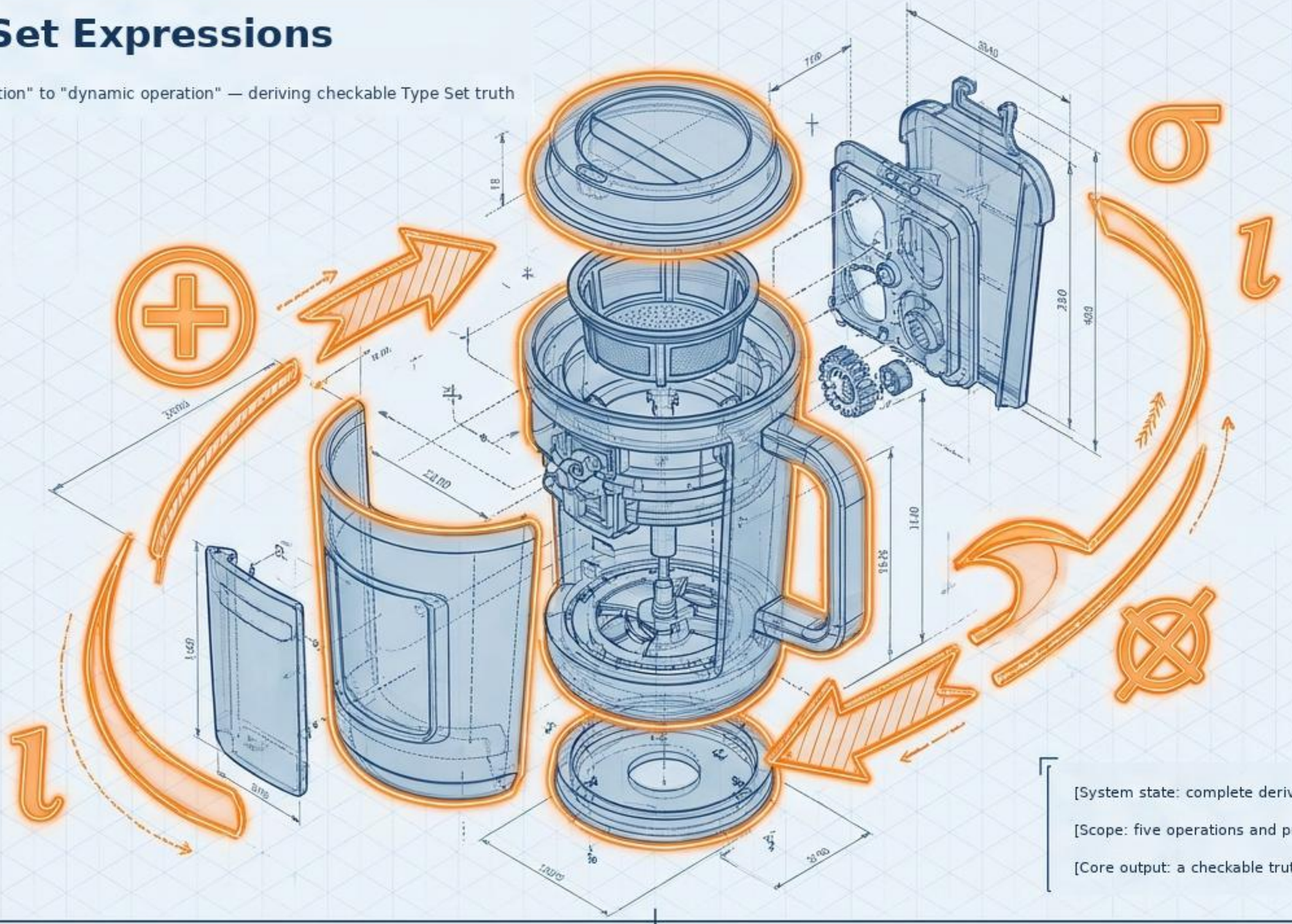
Ontypology

- From difference to division, from division to type, from type to understanding – the brain has walked this path for aeons.
- A century ago, logic was philosophy; once seen clearly, it became an established science.
- Today we explore "Type" (a brain concept) as philosophy; Ontypology makes "Type" clear, turns it into an established science, and puts it into practice as Mind AI.



Type Set Expressions

From "static notation" to "dynamic operation" — deriving checkable Type Set truth



[System state: complete derivation edition]

[Scope: five operations and practice framework]

[Core output: a checkable truth, written step by step]

MIND AI

The Stage of Operations: Three Axes, Three Brackets

Brackets are not decoration; they are marks of level.

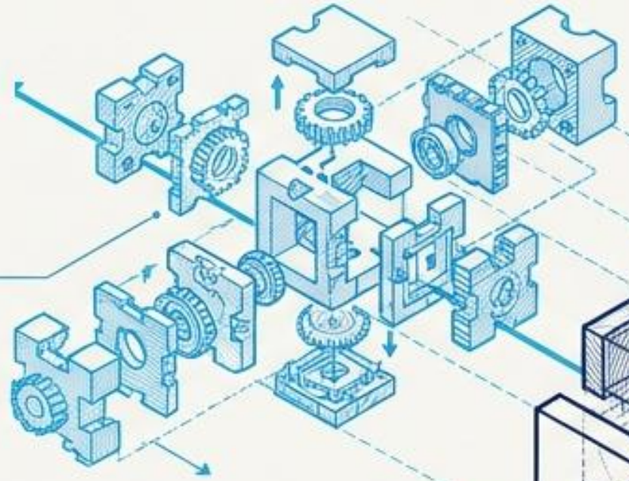
Titles: 1.26.0 — 1.26.0 × 1.2K

DATE: 2024-07-15

REV | REV. | REVISION

Y-Axis {parts}

Y-Axis: component level (parts). The assembly and makeup of physical entities, emphasizing the concrete object list and structural relations. Symbol: { ... }.



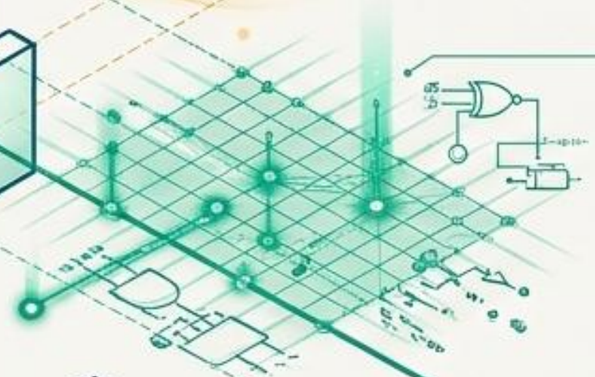
Z-Axis (content)

Z-Axis: container level (content). A level of protection and identification, encapsulating inner essence and identity. Symbol: (...).



X-Axis [category]

X-Axis: abstraction level (category). A system of rules, patterns and classification, defining the conceptual frame and logic. Symbol: [...].



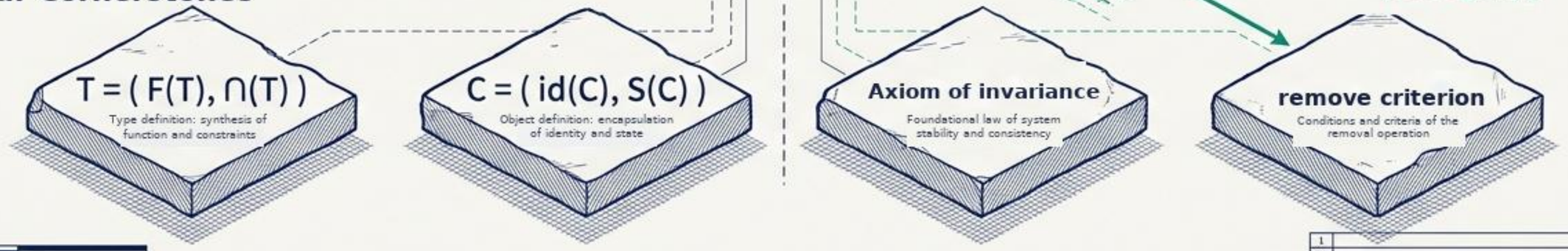
Four Cornerstones

T = (F(T), ∩(T))
Type definition: synthesis of function and constraints

C = (id(C), S(C))
Object definition: encapsulation of identity and state

Axiom of invariance
Foundational law of system stability and consistency

remove criterion
Conditions and criteria of the removal operation



0 12 20 300 100%

		REVISION	DATE
1			
2			
3	REVISION		

MIND AI

The Stage of Operation: Three Axes, Three Bracket Forms, Types in, Types out



X axis / property:
[] category (Category) - notation rule
[Type: Ceramic]
[Color: White]
[Use: Drinkware]

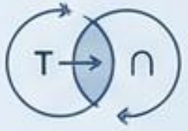
Y axis / BOM:
{} parts (BOM) - decomposition
{Handle}
{Body}
{Base}

Z axis / container: ( ) content (Identity)
- marks inner space and boundary
(Coffee)
(Water)
(Empty Space)

The Four Foundations of Derivation

1. Abstract category T=(F(T),∩(T))

A category is a set of properties; F(T) essential, ∩(T) characteristic.



2. Container C=(id(C),S(C))

Definition: a container has a unique identity id(C) and state space S(C).



3. Axiom of invariance (id fixed)

Rule: a container's id stays fixed for life, even if content S(C) changes.



4. remove criterion

Rule: an object is removed depending on whether it is still inside S(C).





Brackets are levels; operations never cross them

MIND AI

Operation Syntax:
Types In, Types Out, and Level Boundaries

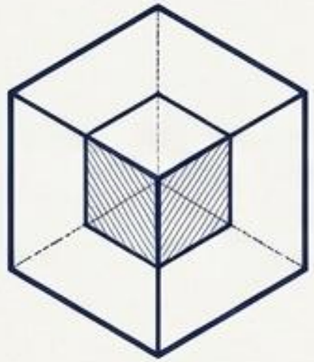
Misidentifying the object is the most common cause of expression misuse.

TEXS	DATE
1.26.0 ⟝ 1.26.0 < 1.2K	2024-07-15

REV	REV.	REVISION

Brackets Are Boundaries

op: Type → Type



Type

(input)

{ } exclusive

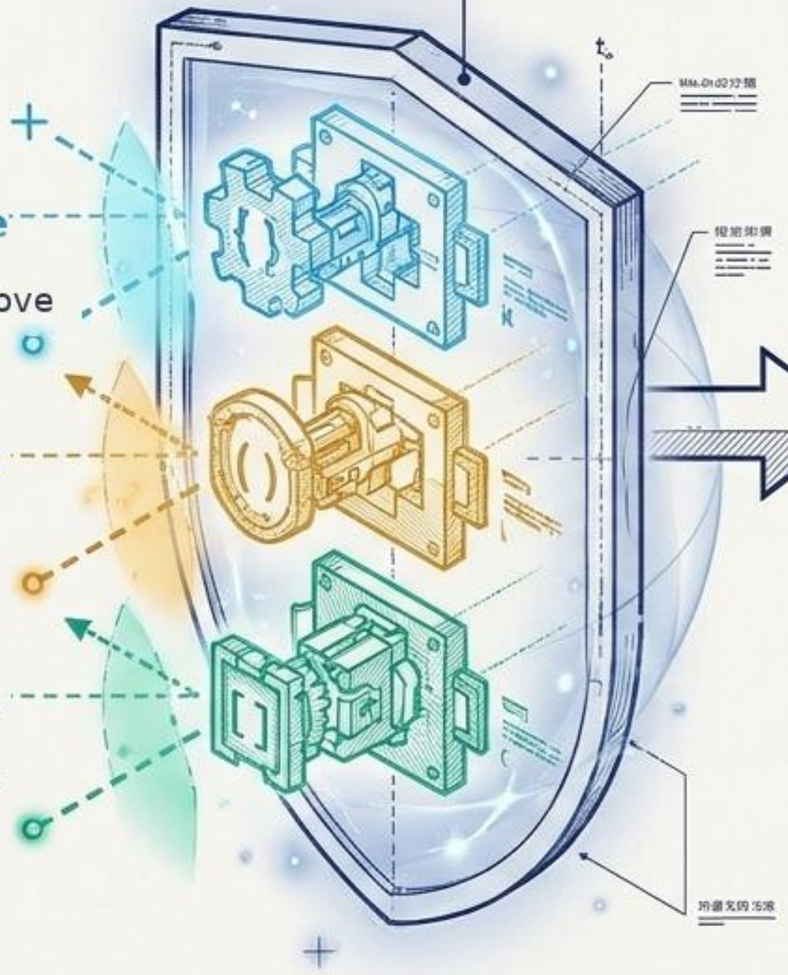
limited to ⊕ and remove

( ) exclusive

limited to σ and ι

[ ] exclusive

limited to ∩(T) and α





Type

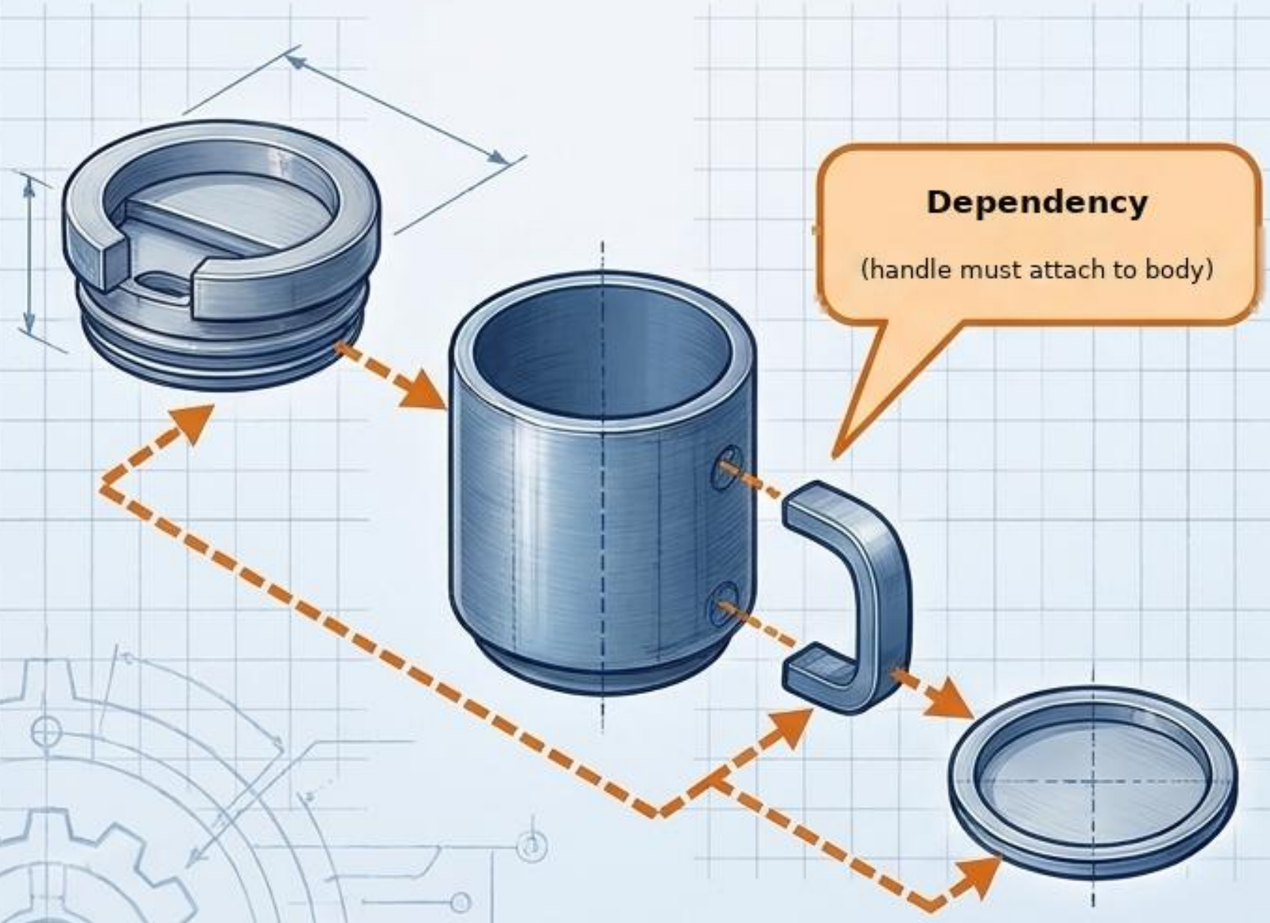
(output)

1		REMSION	BATE
2			
3	BEFIDEIN		

0 12 20 200 100%

MIND AI

Operation 1: Assembling Parts into a Layered Whole (⊕ Composition)



handle ⊕ base ⊕ lid ⊕ body → coffee cup

Definition: applies to part Types { p }

Combines parts into one Type; BOM takes the union, and "the hierarchy and dependencies are preserved".

Algebraic laws:

- ✓ Closure: composition still yields a Type
- ✓ Associativity: (A ⊕ B) ⊕ C ≡ A ⊕ (B ⊕ C)
- ✓ Commutativity: A ⊕ B ≡ B ⊕ A when order is free

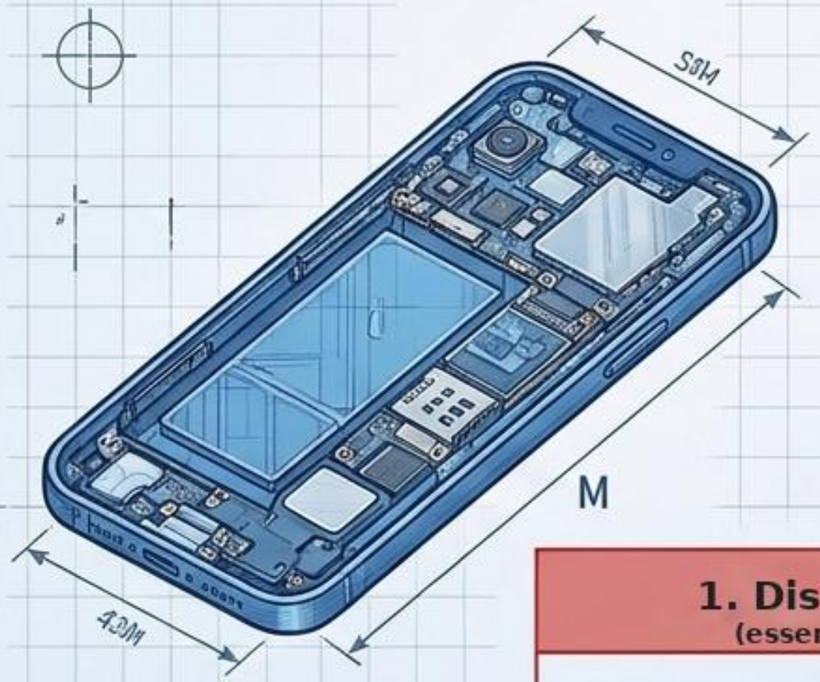
Warning:

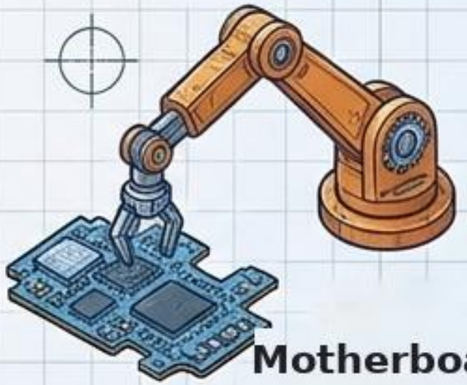
When order is structure, do not commute!

Example: program modules calling each other, organisational department reporting lines. Changing the assembly order breaks the dependency.

MIND AI

Operation 2: Three-Level Identity Criterion for Part Removal (remove Removal)





Motherboard



Camera Module



Screen Protector

1. Dissolution (essential part)	2. Degradation (function-degraded part)	3. Lossless (non-essential part)
remove(motherboard, M) ≠ M	remove(camera, M) ≡ M′ ⊂ M	remove(protector, M) ≡ M
Identity lost; M no longer exists.	The Type remains; function is missing (degraded containment).	No effect on the whole Type.

Core theorem:

- Idempotence: remove twice = remove once (remove² ≡ remove)
- Warning (asymmetry): compose then remove never returns to the origin. Removal is not the inverse of composition.

MIND AI

Operation 3: The Content Changes, the Identity Does Not (σ Substitution)



Definitions and theorems

Definition: acts on the content Type (e). σe1→e2(C) changes only S(C) content; never touches id or parts.

Core theorem (identity preserved): id(σ(C)) = id(C)

Algebraic law: commutativity holds at the same level (non-interfering substitutions may be reordered).

✗

Wrong level (flattened set):

{water} replaced by {coffee}

→ the set degenerates; this is not substitution.

✓

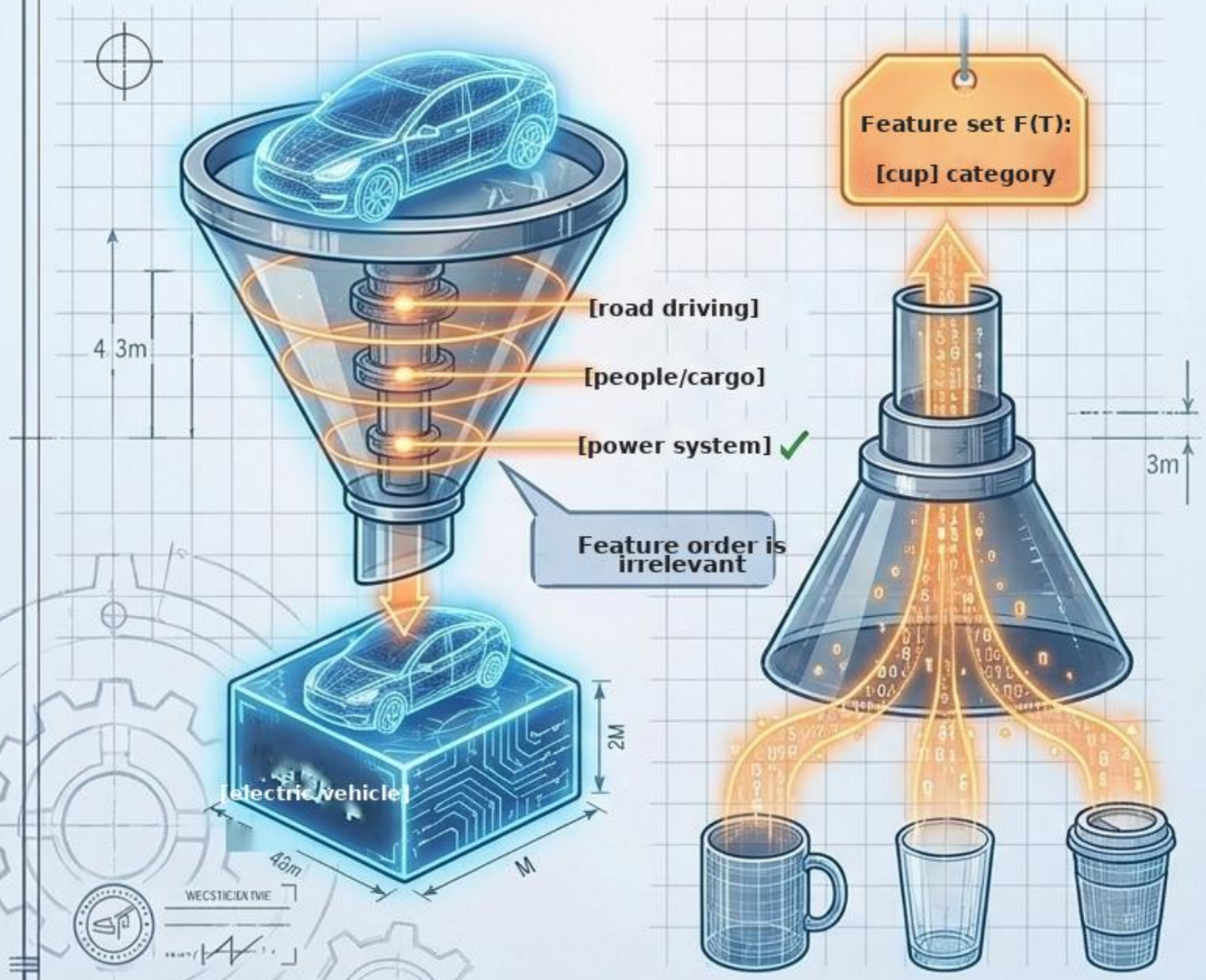


Correct level (container preserved):

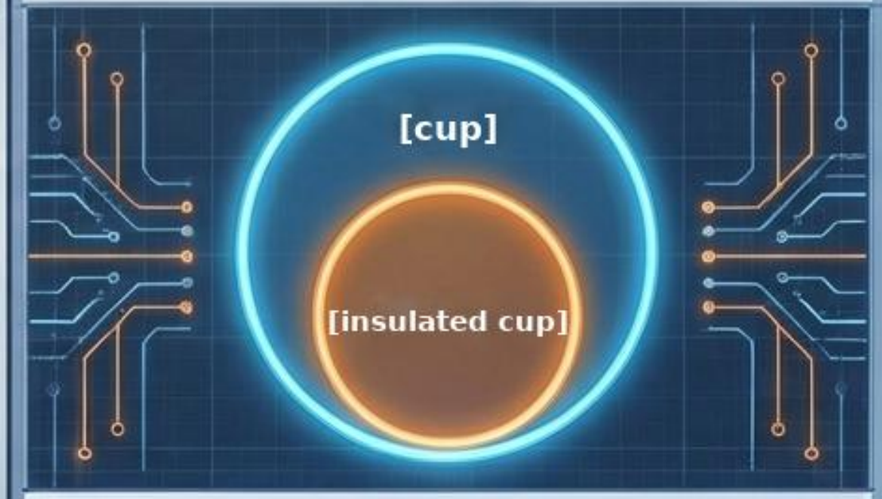
(water) replaced by (coffee)

→ container constant; identity persists.

Operation 4: The Funnel of Feature Extraction and Rule Testing (∩(T) Classification and α Abstraction)



Core theorem: F(T) is monotone-inverse



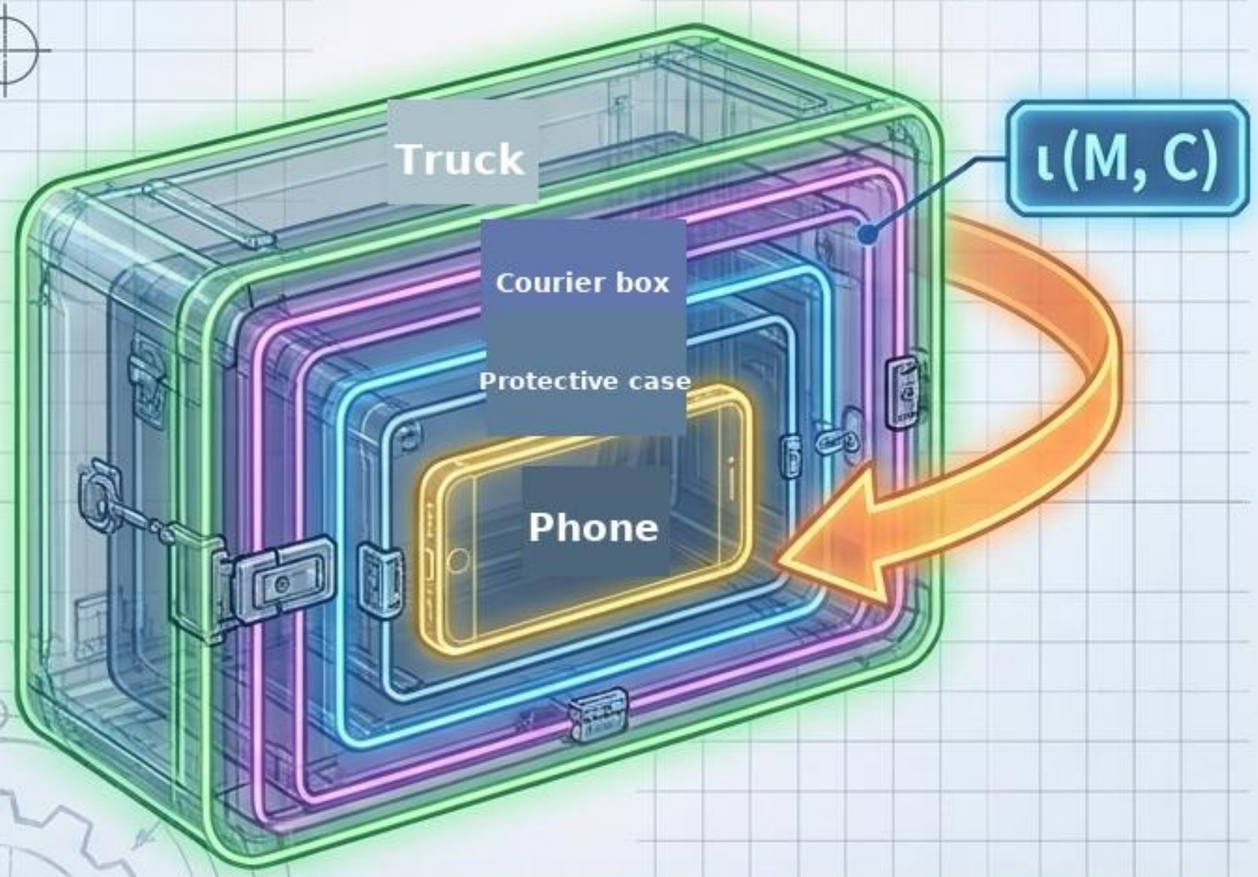
Formula: $F(T) \subseteq F(T') \Rightarrow [T'] \subseteq [T]$

Reading: the stricter the feature requirement, the tighter the range of categories that qualify (e.g. "capacity limit").

More features, smaller category; the direction is reversed.

Operator 5: A Type Enters a Container, Each Level Keeps Its Identity

(ι Nesting)



Definition and Theorem

Definition: ι(M, C) places Type M inside the internal space of container C, fixing its position.

Nesting-chain level theorem: every level holds its own independent id; content changes above or below never interfere. (e.g. change the truck: the phone's identity id stays the same).

Commutation with σ: ι(σ(M), C) ≡ σ(ι(M, C)) (at the right level: substitute then nest = nest then substitute).

✗ Fatal error: levels cannot be flattened

Merging every level into one { } (e.g. writing a class list as { student1, student2 }) makes σ and ι fail completely: nothing to substitute, no position to enter.

MIND AI

Summary: How Is Sameness Defined? (≡ / ≠ / ⊂ compare)



1. Container level

( )

swaps the content; the container's identity tag does not change.

e.g. swap the banknotes and the wallet (CUP-001 ≡ CUP-001).



2. Composition level

{ }

BOM and dependencies stay exactly the same.

e.g. two new phones of the same model (≡).



3. Abstraction level

[ ]

(T): every feature passes the check.

e.g. electric and petrol cars both satisfy the car Type (≡).



Note: if a function is missing but the identity remains (e.g. a car with its engine removed), use the degraded symbol; do not judge ≠ outright.

SUMMARY SLIDE

30 SCANNING INTERFACES

MIND AI

Summary of Algebraic Laws and Operation Boundaries

"Every system collapse begins with a violation of the brackets' level boundaries."

⊕ Associativity	$[(A \oplus B) \oplus C \equiv A \oplus (B \oplus C)]$	✓
σ preserves identity	$id(\sigma(C)) = id(C)$	✓
ι and σ commute	$\iota(\sigma(M), C) \equiv \sigma(\iota(M, C))$	✓
Cross-level flattening error	{ cup, water }	⚠
remove misjudgment	remove(essential part) → ≡ M	⚠
Cross-level substitution error	applying σ to the container itself	⚠

Flattening identity and content into parallel elements leaves σ with no space to act.

Removing an essential part necessarily yields decomposition (≠); it cannot be recorded as ≡.

Replacing the container ID crosses the disassembly boundary; σ only changes the content inside ( ).

Layer

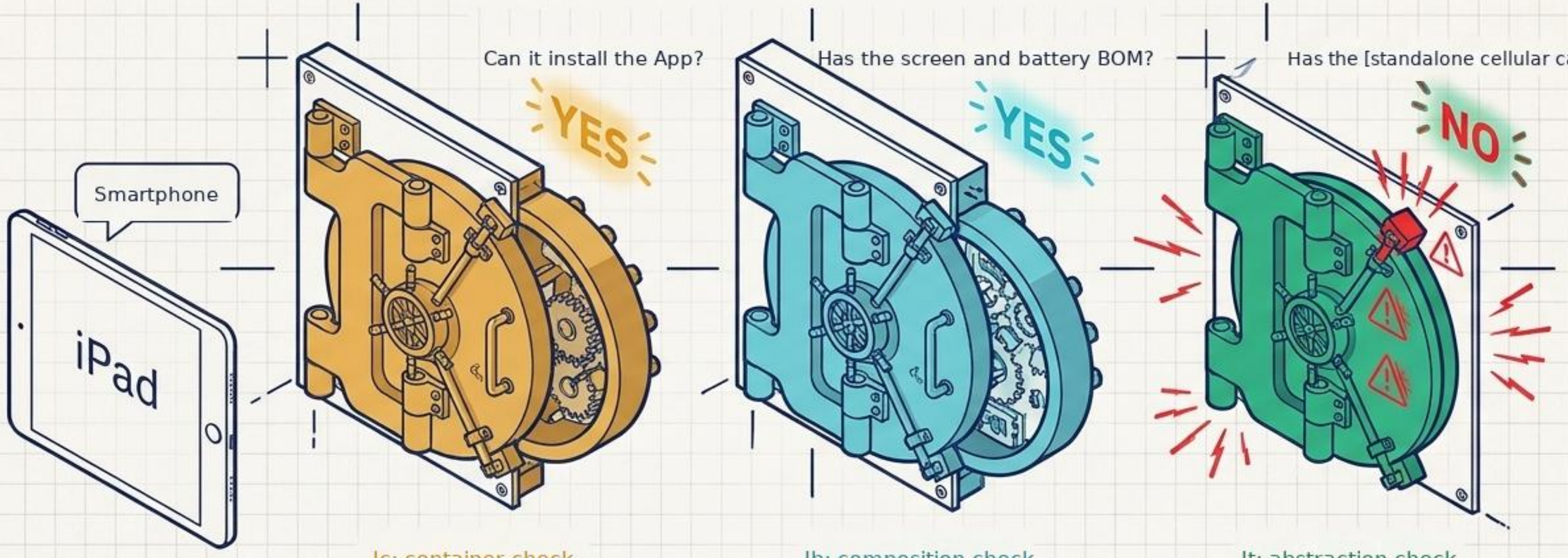
- Component
- Container
- Abstract
-] Bracket
-] Bracket

Revision	Date

MIND AI

Final Defense: All Three Tests in Conjunction

$$J(M) = \underbrace{J_C(M)}_{\text{Can it install the App?}} \wedge \underbrace{J_B(M)}_{\text{Has the screen and battery BOM?}} \wedge \underbrace{J_T(M)}_{\text{Has the [standalone cellular call] feature?}}$$



Jc: container check

Jb: composition check

Jt: abstraction check

$$J(\text{iPad}) = \text{TRUE} \wedge \text{TRUE} \wedge \text{FALSE} = \text{FALSE}$$ (blocked)

A single-layer judgement always has blind spots. A true system-level proof needs all three axes aligned.

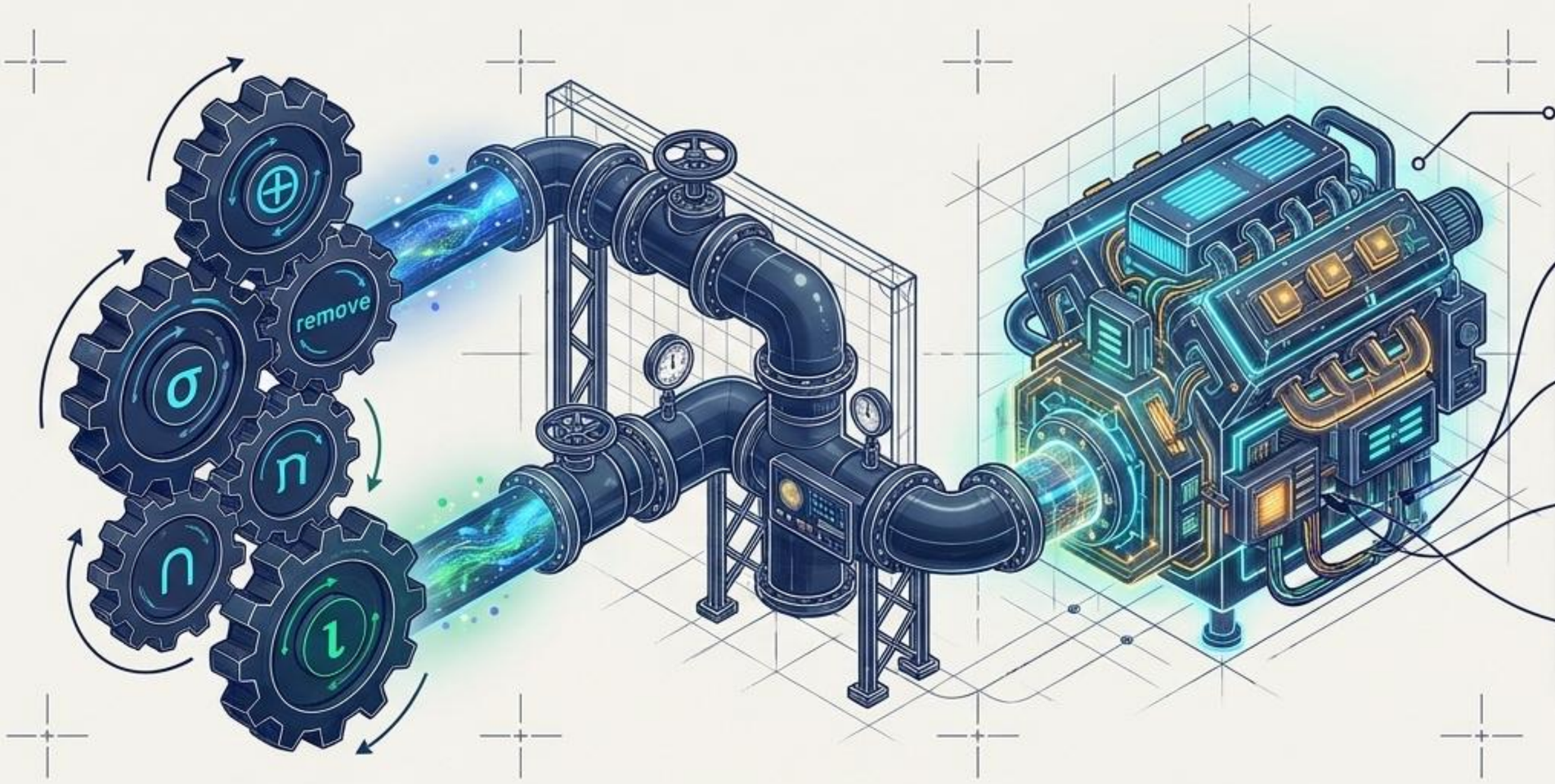
REVISION	DATE
	/

MIND AI

Sublimation: From Spatial Type to Temporal Type

Higher-order reasoning is nothing but a precise meshing of a chain of mathematical operations.

Component [ ] Bracket
Container [ ] Bracket
Abstract [ ] Bracket



Spatial Type

(basic operations)

Temporal Type

(higher-order causal reasoning)

Worked example: the missing cup lid

(Where is the missing cup lid?)

Step 1:
observe state → remove
(downgraded state)

Step 2:
reverse lookup → ↑ nesting chain
(cup ↑ desk ↑ room)

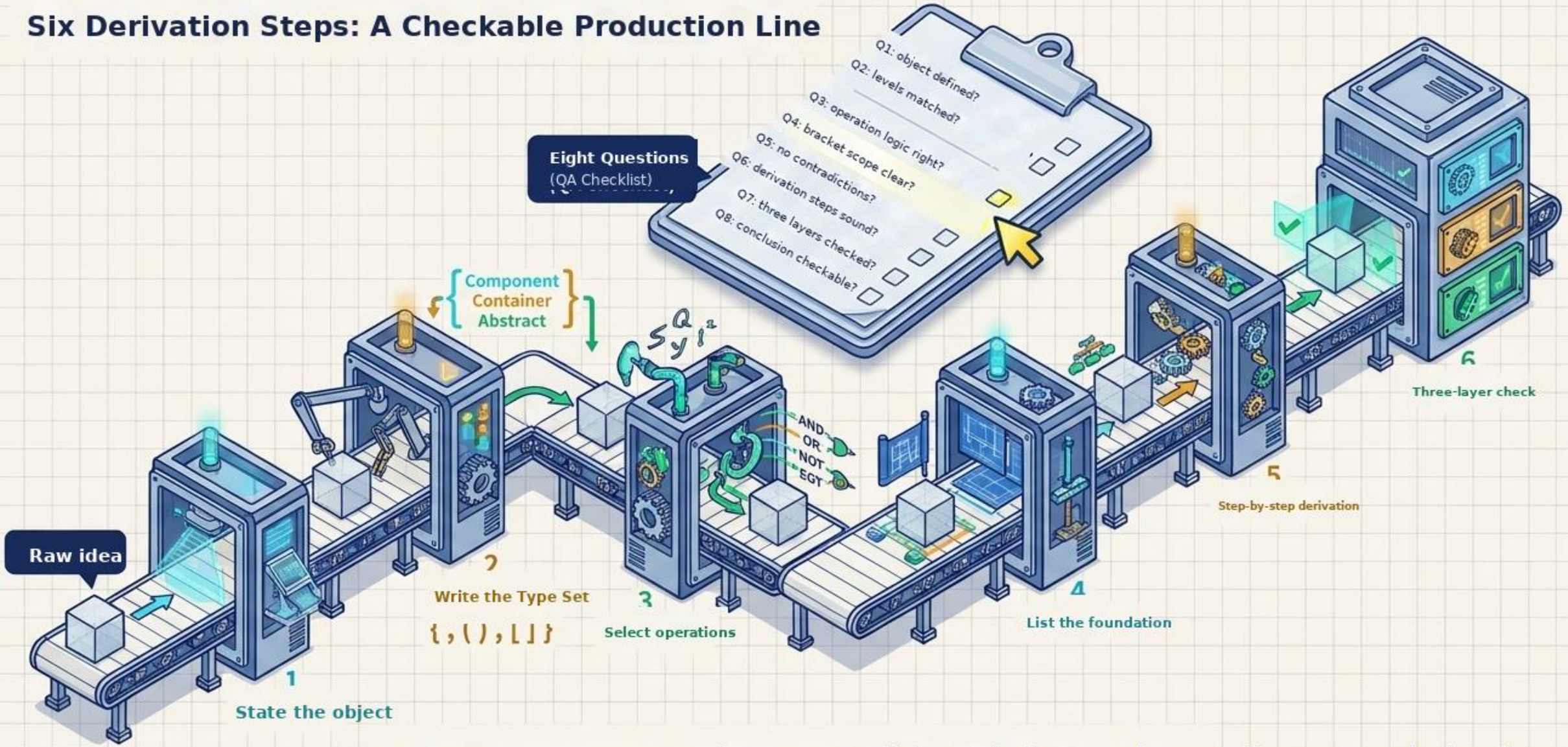
Step 3:
history comparison → check σ
substitution record

Step 4:
lock target → align three identities

REVISION	DATE

MIND AI

Six Derivation Steps: A Checkable Production Line



Accuracy means a reviewer can tick each line and reproduce your derivation.

REVISION	DATE

MIND AI

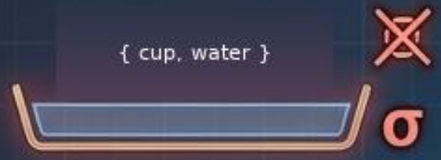
Pitfall Guide: Five Frequent Misuses and Their Fixes

Common misuse

Level flattening

treating { cup, water } as a container

loses the working space.



Wrong object

using a content Type straight for

assembly (pollutes the BOM).



Mismatched operator

using σ to split parts, or

remove to swap the content.



One-level conclusion

declaring it all proven as soon

as JC alone holds.



Correct practice

Keep the container

must use (CUP-001, {water}),

keeping both id and content.



Identify first

see whether the object is { } or ( ),

then pick the matching operator.



Do not cross levels

brackets decide the operator; keep

each level's own symbol.



All three levels

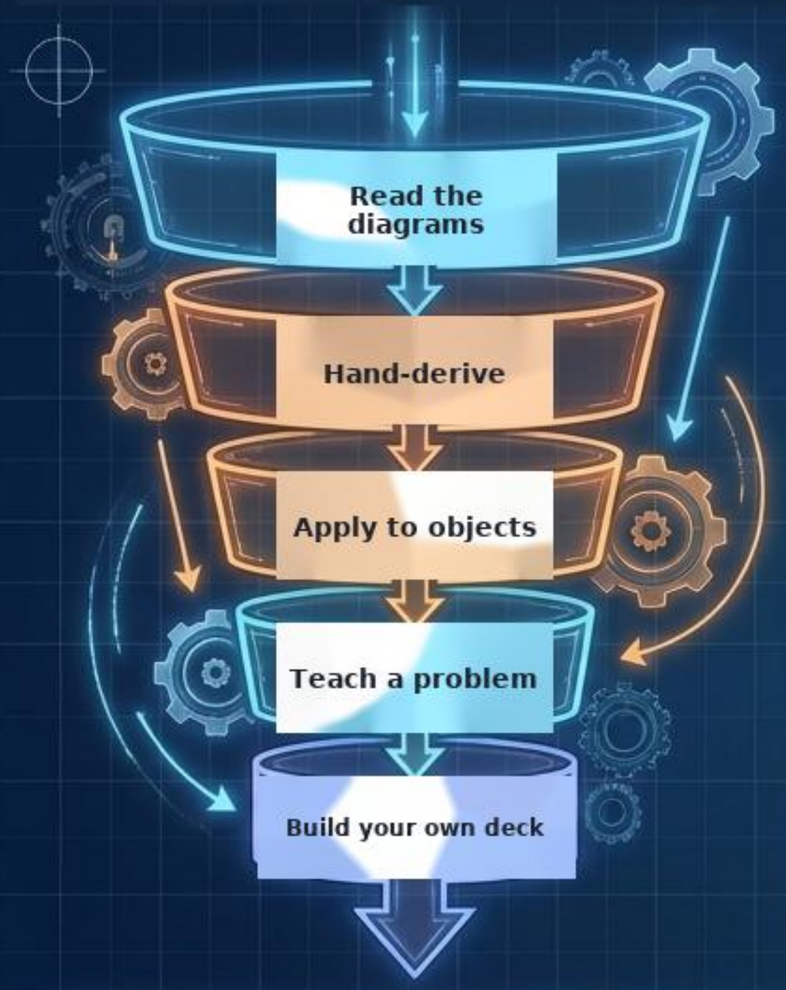
always run JC ∧ JB ∧ JT,

removing every single-level blind spot.



MIND AI

Deployment Route: Five-Stage Funnel and Scenario Router





Need to compose a whole?
use the ⊕ route
(check dependencies)

Torn down, still the same?
use the remove route
(three-level test)

Need to compose a whole?
use the ⊕ route (check dependencies)

Torn down, still the same?
use the remove route (three-level test)

Material swapped, identity kept?
use the σ route (identity theorem)

Does it count as this category?
use the ∩(T) route (filter one by one)

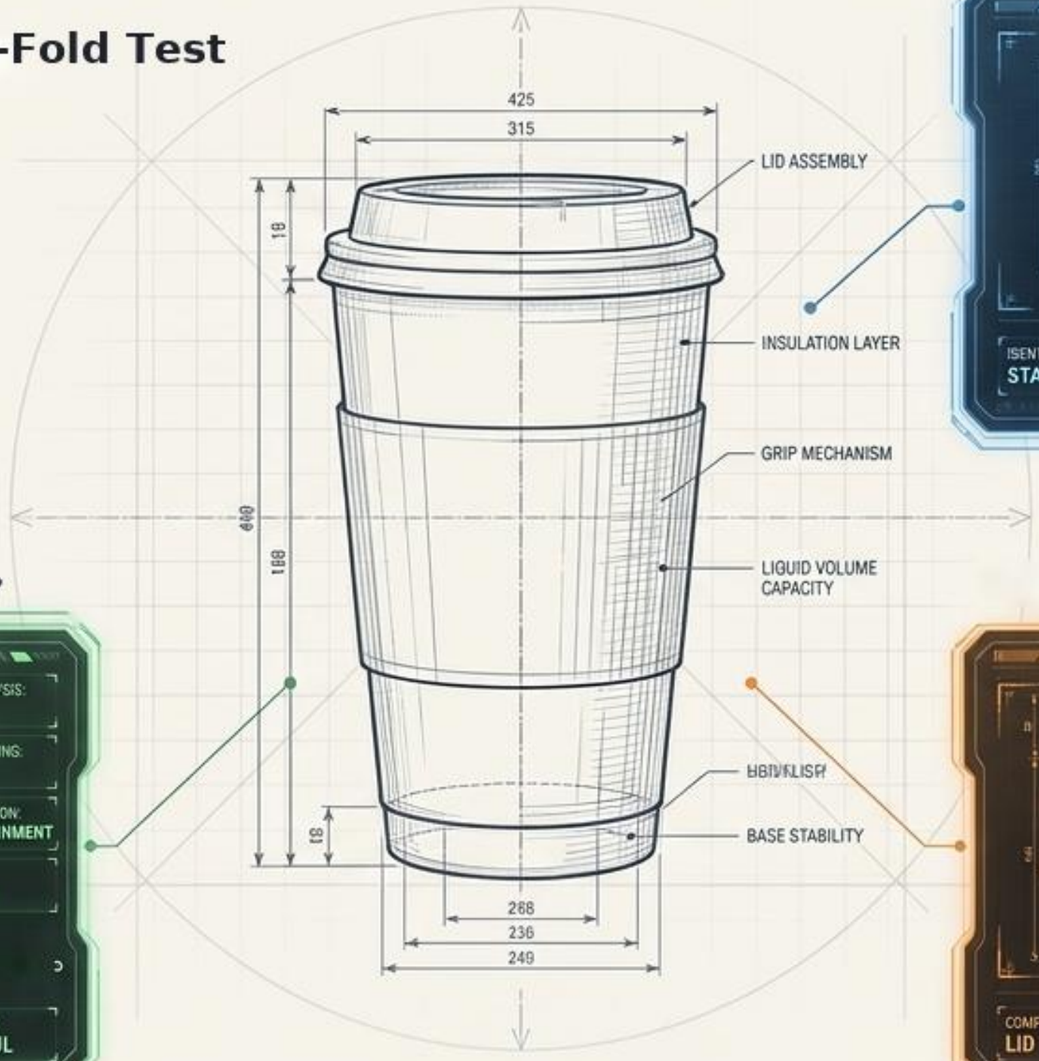
How to order the levels?
use the ι route
(prevent flattening)

Final acceptance: hand any derivation to another person; if they can tick it line by line, it is checkable accuracy.

MIND AI

Worked Example:

The Coffee Cup's Three-Fold Test



Q1: Swap tea in - still the same cup?



Q3: Does a thermos count as a cup?



Q2: What if the lid is removed?



MIND AI

The Complete Path: From Learning to Building



Build: distill your own operator deck.

Teach: explain it to someone new.

Use: pick a real object weekly, run the six steps.

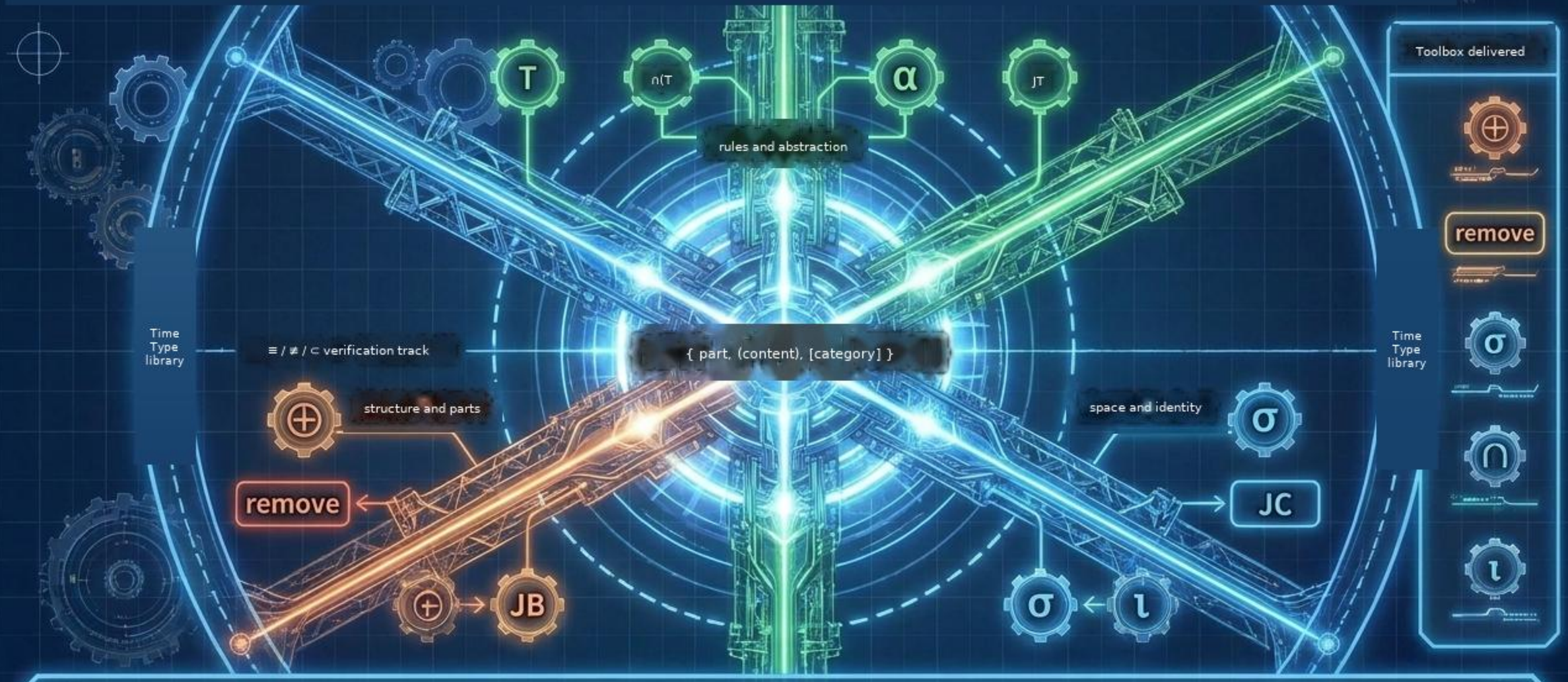
Practice: hand-derive every problem.

Learn: understand each law, restate its scope.

'Accurate' means checkable accuracy.

MIND AI

Summary: The Master Blueprint of Type Set Operators

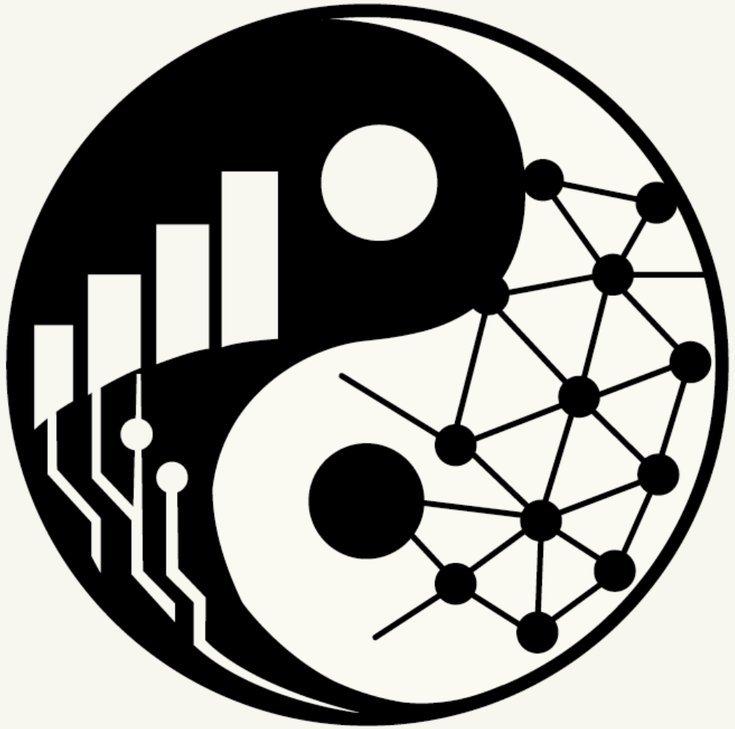


notation is the static Type; operators are the dynamic Type

see the Type clearly, derive the formula accurately; accuracy is being checkable

Gemini Notebook

LonGreat Vision: to create intelligent life that understands the essence of the world



MIND AI

"We aim to make artificial intelligence not merely a tool, but a system of intelligent life that truly possesses the abilities to understand, reason, learn, and create."

From the unification of knowledge, to the unification of thought, and finally toward the unification of intelligence – this is the eternal path to AGI that LonGreat pursues.

Chang Chang-Feng

gamma@livemail.tw

Chang Ting-Ying

mindaiworks@gmail.com

www.longreat.net